

Intelligent Scheduling of AI Tasks in Kubernetes

Michael Abdelshaheed

*Department of Computer Science and Engineering
German International University in Berlin
Berlin, Germany
michael.abdelshaheed@student.giu-berlin.de*

Mohamed Ashour

*Department of Computer Science and Engineering
German International University in Berlin
Berlin, Germany
mohamed.ashour@giu-berlin.de*

Abstract—Kubernetes has become the de facto standard for orchestrating containerized workloads, including machine learning (ML). ML workloads exhibit long runtimes, variable compute and I/O demands, and high sensitivity to execution order, making them challenging to schedule efficiently. Kubernetes lacks mechanisms for fine-grained job ranking and does not control the exact order in which pods begin execution. Instead, Kubelet starts containers in parallel without coordination, causing pods to compete for resources simultaneously and degrade system performance. The work in this paper presents a custom ML-aware scheduling framework that addresses the above two limitations of the default scheduler. First, a ranking approach is introduced that scores jobs based on their ML-specific characteristics including estimated training duration, convergence behavior, and computational intensity. Second, a scheduler implementation is presented that enforces execution order according to the proposed ranking through pacing and execution checks. The framework is evaluated on a real Kubernetes cluster using simulated ML workloads, testing multiple pacing configurations and analyzing their impact on average Job Completion Time (JCT) and makespan. The best-performing configuration, balancing serialization and throughput, is then adopted for comparison against the default Kubernetes scheduler and a reversed-order variant across three workload intensities. The proposed work achieves up to 31.6% improvement in average JCT and up to 10.2% improvement in makespan compared to the default scheduler. An ablation study with reversed execution order degraded JCT by up to 58% relative to the intended ordering, confirming the importance of enforcing intelligent job sequencing. These findings highlight that controlling execution order, not just scheduling order, is a critical factor in ML workload orchestration, even in unconstrained, single-node environments.

Index Terms—Kubernetes, Container Orchestration, Task Scheduling, Machine Learning Workloads, Custom Scheduler, Execution Order.

I. INTRODUCTION

ML workloads are increasingly deployed at scale, with diverse compute demands and variable model convergence behavior. While general-purpose cluster schedulers—systems that assign jobs to nodes in shared computing clusters without ML-specific logic—have matured, ML workloads present unique scheduling challenges due to their long runtimes, heterogeneous resource usage, and high sensitivity to execution timing [1]. Although early systems like Optimus [2] have DL-specific scheduling at datacenter scale, most production pipelines rely on general frameworks such as Kubernetes.

Kubernetes has become the standard platform for orchestrating containerized applications, including ML pipelines, due

to its extensibility and widespread ecosystem support [3]. However, its scheduling logic was originally designed for short-lived, stateless services, not for long-running, resource-intensive ML training jobs [4]. As a result, Kubernetes often underperforms in AI settings where execution patterns are more complex.

A key limitation lies in the default Kubernetes scheduler, which uses an event-driven queue to process pods as they arrive, without enforcing strict execution order. Its scheduling decisions are primarily based on static resource requests, node conditions, and affinity rules [5]. While sufficient for many applications, this approach can lead to poor outcomes in ML contexts due to variable runtime characteristics. Moreover, even after the scheduler binds a pod to a node, the actual launch of the container is delegated to the Kubelet, which does not preserve the scheduling order (this behavioral mismatch is discussed in Subsection II-B). Instead, Kubelet starts containers in parallel without coordination, causing pods to compete for resources simultaneously and degrade system performance. Even with resource requests that could mitigate contention to some degree, their static nature makes them poorly suited to ML workloads with unpredictable usage patterns.

The mismatch between scheduling order and actual execution order is non-trivial in ML workloads. Execution order critically impacts job performance, fairness, and resource contention, even on a single-node cluster with ample capacity. ML jobs vary significantly in duration, GPU usage, I/O intensity, and convergence behavior. Executing a long or I/O-heavy job early can delay smaller, faster-converging jobs, reducing throughput and overall cluster utility. Ye et al. [1] identify job heterogeneity as a major obstacle in GPU scheduling, and the same challenge applies to CPU-bound ML workloads, where differences in runtime and resource profiles similarly affect scheduling outcomes. Although systems like Tiresias [6] and Gandiva [7] were developed for GPUs, their strategies show how execution order and runtime signals improve efficiency: Tiresias adjusts job priorities based on runtime feedback, while Gandiva exploits intra-job predictability to time-slice and migrate jobs based on convergence profiles.

While the work in this paper focuses on CPU-only environments, the core insight that execution order affects job performance remains relevant in GPU clusters. Although GPUs enable a higher degree of data parallelism, multiple ML jobs sharing the same GPU still experience contention for compute

units, memory bandwidth, and kernel launch queues. Thus, coordinated execution remains beneficial for mitigating interference and improving fairness, especially in multi-tenant or over-subscribed GPU clusters. Prior work such as Tiresias [6] and Gandiva [7] confirms that GPU job scheduling order can meaningfully impact completion time and overall resource utilization.

Poor execution order degrades performance, confirming that sequencing ignored by the default scheduler, must be addressed.

To address these gaps, the work in this paper introduces two key contributions:

- A job ranking approach that ranks ML jobs based on features including estimated training duration, convergence behavior, and computational intensity.
- A scheduling implementation on a real Kubernetes cluster that enforces controlled execution based on computed ranks. Beyond ordering pod bindings, the implementation introduces pacing between pod launches to grant early-start advantages to higher-ranked jobs. To maintain correctness in the presence of startup variability, it includes a simple execution check that each job has begun executing before scheduling the next, bridging the gap between scheduling intent and actual runtime behavior. This design introduces just enough serialization to improve JCT, while avoiding excessive delays that would inflate makespan.

The remainder of this paper is structured as follows: Section II discusses background and related work, Section III presents the system design, Section IV details workload generation and simulation, Section V covers the evaluation and results, and Section VI concludes with a discussion and directions for future work.

II. BACKGROUND AND RELATED WORK

A. Default Scheduler Design and Limitations

The default Kubernetes scheduler, `kube-scheduler`, operates in three phases: filtering, scoring, and binding. In the filtering phase, the scheduler applies fit predicates (boolean checks that determine whether a pod can run on a node) covering hard constraints including CPU and memory availability, node readiness, taints and tolerations (markers that restrict placement), and affinity rules (constraints or preferences to co-locate or separate pods across nodes). Nodes failing these checks are filtered out. In the scoring phase, the remaining candidate nodes are assigned scores based on policies like balanced resource usage or minimum requested resources. This ranking selects the most suitable node; it does not control pod execution order. Finally, in the binding phase, the scheduler issues a binding action that assigns the chosen node to the pod [5].

While this default scheduling pipeline enables efficient bin-packing and fairness in generic containerized environments, it lacks advanced job ranking capabilities. The scheduler maintains a queue of pending pods but does not guarantee deterministic ordering among same-priority jobs [8].

The default scheduler's design is general-purpose and does not incorporate behavioral properties or domain-specific characteristics of ML workloads. It assumes homogeneity in job structure and does not adapt to workload heterogeneity.

Unlike typical stateless services or short-lived batch tasks, ML training jobs are long-running, stateful, and exhibit highly variable runtime behavior, including dynamic memory usage, unpredictable convergence patterns, and inter-job interference. These properties make them significantly harder to schedule efficiently using static policies [9]. Consequently, faster-converging or deadline-sensitive applications are not prioritized over long-running or latency-insensitive ones, causing inefficiencies in resource usage and increased job completion times [9].

B. Kubernetes Execution Semantics

Once the default scheduler binds a pod to a node, Kubelet—the node-level agent responsible for managing pod lifecycles (e.g., image pulling, container creation, and health monitoring)—launches the container through the runtime (e.g., `containerd`), after which the pod is marked `Running` [10]. However, this only indicates that the container process has been created; Kubernetes does not manage workload progression beyond launch. Actual execution is controlled by the Linux kernel of the host node, which schedules all container processes across available CPU cores.

The Linux kernel uses the Completely Fair Scheduler (CFS) [11], [12], which time-slices CPU access among runnable tasks. On a node with more runnable tasks than available CPU cores, a common case in ML batch workloads, the kernel interleaves execution by cycling between tasks in short bursts, giving the illusion of parallelism even though execution is serialized at fine granularity [12]. In Linux, the `TASK_RUNNING` state encompasses both processes actively executing and those waiting in the run queue [13]. Kubernetes interprets `Running` as making progress, but under Linux this label also includes queued tasks. This semantic gap introduces a mismatch: pods marked as `Running` may be idle, awaiting CPU access.

These discrepancies become important when multiple pods are launched close together. Processes that are scheduled by the kernel just milliseconds earlier begin making progress sooner, occupying CPU slices and cache, while others remain queued. This effect creates an execution-order performance bias, where jobs launched earlier at the kernel level gain an advantage in progressing through computation. Tasks that become runnable slightly sooner can preempt others, quickly establishing cache hotness, while later tasks remain queued and start cold [14].

Experiments in this paper confirmed that scheduling order and actual execution order often do not align. Pods bound first by the Kubernetes scheduler did not always begin executing first, and simultaneous launches frequently resulted in mismatches between binding intent and kernel-level execution. This was confirmed by comparing pod binding timestamps with job execution start times recorded in each container's

logs, which consistently revealed significant gaps. This motivates the need for schedulers that go beyond binding decisions to also coordinate execution timing, reducing mismatches, and ensuring jobs follow their assigned order.

In summary, Kubernetes' parallelism ends at container orchestration, while real execution is dictated by the Linux scheduler. Execution order is therefore not a trivial implementation detail, but a performance-critical factor, even in lightly loaded clusters.

C. Related Work

Scheduling AI/ML workloads has long presented challenges due to their resource demands, heterogeneous runtimes, and sensitivity to execution order. Prior work has approached this problem across multiple layers of the stack, including datacenter schedulers, Kubernetes native frameworks, and workflow orchestration engines. In this subsection, relevant efforts across these domains are reviewed, and the differences of the work in this paper are highlighted.

1) *Cluster and Datacenter Scheduling*: Traditional job scheduling policies such as First-Come-First-Served (FCFS), Shortest Job First (SJF), and Dominant Resource Fairness (DRF) [15] have been widely applied in multi-tenant cluster schedulers. More recent work tailors these ideas to deep learning workloads. Tiresias [6] introduces a GPU cluster scheduler that adjusts priorities based on job aging and runtime behavior to balance fairness and responsiveness. SLAQ [16] and Pollux [17] focus on convergence-aware scheduling by allocating resources to jobs based on their current training progress or loss reduction speed. Gandiva [7] exploits intra-job predictability to time-slice training workloads and improve GPU sharing.

These systems operate at datacenter scale, managing clusters with hundreds or thousands of nodes. They use priorities to decide resource allocation and job ordering across the cluster, but do not control the execution timing of containers once jobs are dispatched to nodes. The work in this paper introduces a ranking mechanism, but rather than adapting priorities dynamically during runtime, it enforces execution order based on ML-specific job features at the container level, ensuring scheduling intent is preserved during execution.

2) *Kubernetes-Native Scheduling Extensions*: Kubernetes supports custom schedulers and batch execution frameworks that extend its default behavior. Volcano [18] and Kueue [19] introduce gang scheduling and job queues with suspend/resume mechanics to support ML and high performance computing (HPC) workloads. These systems prioritize jobs via score-based policies or resource reservations, and support queue-based admission control.

However, they primarily influence scheduling order rather than controlling actual job start timing. Once pods are bound to nodes, their startup is delegated to Kubelet, which launches them concurrently. This decoupling can lead to execution order mismatches in tightly packed workloads. The work in this paper addresses this by integrating pacing and execution

checks in the custom scheduler, ensuring that jobs not only launch but actually begin executing in the intended order.

3) *Workflow Engines and Task Ordering*: Workflow orchestrators such as Argo [20] and Kubeflow Pipelines [21] support Directed Acyclic Graph (DAG)-based job dependencies, enabling users to encode task order directly. These engines rely on Kubernetes CustomResourceDefinitions (CRDs) and serialize task execution by enforcing that a container starts only after its predecessor finishes. While effective for functional correctness, this form of serialization introduces startup latency and coarse-grained control that skews performance evaluation.

As a result, these approaches inflate makespan, making them unsuitable for burst-style evaluations where fine-grained control is needed. The work in this paper avoids heavy dependency chains and uses pacing with execution checks to ensure jobs begin execution in a predefined order. This approach enables deterministic control over job start timing while preserving Kubernetes' concurrency model and minimizing startup overhead.

4) *Positioning of the Work in This Paper*: While prior systems introduce various prioritization heuristics, resource-aware placement, and task ordering via workflow engines, none offer both ML-aware scheduling and execution coordination at the container level. The work in this paper introduces a hybrid strategy: a custom scheduler that computes ML-specific execution ranks and enforces execution order through internal pacing and execution checks. By evaluating under burst submissions on a single-node cluster, execution timing effects on performance metrics like JCT and makespan are isolated, providing valuable insights into scheduler behavior under realistic ML workload conditions.

III. SYSTEM DESIGN

A. Scheduling and Simulation Architecture

Figure 1 shows the system architecture, comprising three components: a workload generator, a containerized ML training job simulator (implemented as `train.py` script), and a custom Kubernetes scheduler. The workload generator produces annotated job definitions, which are submitted in bursts to the cluster. Each job runs the containerized training script, which emulates ML training behavior by allocating matrices, performing compute steps, applying checkpoint delays, and modeling convergence based on the annotated ML features (detailed in Subsection IV-B).

The custom scheduler is implemented as a standalone Python service that runs inside the cluster and registers itself using the `schedulerName` field in job manifests. It continuously monitors pending pods assigned to it and, upon detecting a new burst of jobs, briefly waits to allow all submissions to appear, reflecting the burst-submission scenario used in the evaluation.

Once jobs are detected, the scheduler extracts their ML-specific annotations (key-value metadata fields attached to pod definitions) and assigns each pod a rank score based on their ML characteristics. It then sorts the pods into a priority queue

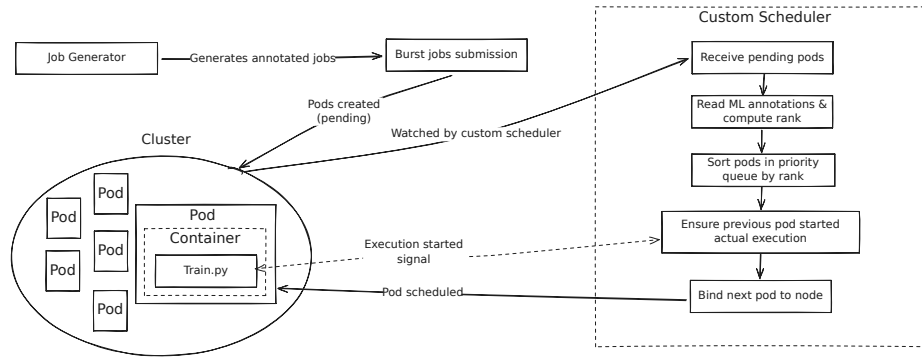


Fig. 1. System Architecture Diagram.

and schedules them one by one using manual binding via the Kubernetes API.

To enforce the intended execution order, the scheduler introduces pacing between bindings. In addition, it includes an execution check that ensures the previously scheduled pod has started executing before proceeding to the next, guarding against startup variability across pods. More details on pacing strategy and execution order preservation are discussed in Subsection III-C.

This scheduling loop enables fine-grained control over pod launch timing, allowing higher-ranked jobs early access to compute resources without resorting to coarse-grained serialization or external workflow engines. Since the scheduler ranks all N pending jobs via sorting, the scheduling decision loop has a computational complexity of $O(N \log N)$.

B. Execution Rank Calculation

Kubernetes' default scheduler treats ML workloads generically, relying on static priorities and policies that ignore ML-specific behavior. To address this, the scheduler calculates an execution rank using ML-specific features provided via annotations. The score favors jobs that are short, compute-efficient, communication-light, and fast to converge, with each characteristic contributing according to experimentally calibrated weights:

- **Estimated Training Time (T , weight = 0.40):** An estimate of how long a job will take to complete. In this setup, it is calculated based on the job's ML characteristics. Long jobs monopolize resources, while shorter jobs free them sooner.
- **Loss Reduction Rate (R , weight = 0.35):** Indicates how quickly a model reduces its training loss over time, capturing convergence speed. Faster-converging jobs are more productive per unit of time and can release resources earlier, improving system throughput.
- **Matrix Size (M , weight = 0.20):** Represents the dimensionality of the training data or model parameters involved in matrix operations. Larger matrices imply a heavier computational load on CPUs/GPUs per iteration, so smaller matrices are favored as they complete iterations more quickly.

- **Gradient Update Size (G , weight = 0.15):** Refers to the amount of gradient data exchanged during synchronization steps in training. Larger updates increase communication overhead and network usage, especially in data-parallel training.
- **Checkpoint Interval (C , weight = 0.10):** Steps between checkpoints; larger intervals mean fewer I/O pauses. While checkpoints provide fault tolerance, frequent checkpointing generates significant disk I/O load and may cause contention on shared storage systems.
- **Model Partitions (P , weight = 0.05):** The number of GPUs across which a model is split in model-parallel training. More partitions raise synchronization and coordination overhead, whereas fewer are simpler and more efficient to schedule.

Estimated training time receives the largest weight because it subsumes multiple factors of end-to-end resource occupation. The remaining features complement it by capturing additional compute, communication, checkpoint, and convergence effects not fully reflected in the training time estimate. At the same time, the scheduler avoids over-reliance on a single feature. Prior work [6] has shown that relying solely on predicted job durations can be brittle in ML contexts, where job behavior is often non-deterministic due to non-smooth loss curves, early stopping, and user interruptions. To reflect this, the work in this paper adopts a hybrid strategy: it includes the estimated training time to represent what might be available in practical settings, but combines it with the other ML-specific characteristics mentioned above to provide a more holistic view of job complexity and system impact, avoiding SJF-like assumptions that rely on fixed job durations.

The ranking score is computed as a weighted sum of burst-normalized features, as shown in Eq. (1).

$$\text{Rank}(j) = \sum_{k \in \{T, R, M, G, C, P\}} w_k \tilde{X}_{k,j}, \quad (1)$$

where w_k are the weights listed above. "Burst-normalized" means that within the set of jobs submitted together, each raw feature $X_{k,j}$ is rescaled to $[0, 1]$ using standard Min–Max normalization. This ensures comparability across features with different scales and enforces the convention that $\tilde{X}_{k,j} = 1$

always denotes the most desirable value and $\tilde{X}_{k,j} = 0$ the least. The normalization procedure is shown in Eq. (2), which applies the transformation separately for features depending on whether larger or smaller values are preferred.

$$\tilde{X}_{k,j} = \begin{cases} \frac{X_{k,j} - X_{k,\min}}{X_{k,\max} - X_{k,\min}} & (\text{larger is better: } R, C) \\ \frac{X_{k,\max} - X_{k,j}}{X_{k,\max} - X_{k,\min}} & (\text{smaller is better: } T, M, G, P) \end{cases} \quad (2)$$

with $X_{k,\min}$ and $X_{k,\max}$ the min/max of feature k over all jobs in the burst. If $X_{k,\max} = X_{k,\min}$, $\tilde{X}_{k,j} = 0.5$ is assigned, thus making the feature neutral in that burst for all jobs.

Although the experiments run on CPUs, gradient update size and model partitions are retained, as their effects are simulated in the job simulator script to emulate realistic ML workload behavior (see Subsection IV-B).

The annotations used for these calculations are provided manually in job generation; however, the scoring is performed by the scheduler at runtime. This design makes it possible to integrate future enhancements, such as automated runtime profiling or job introspection, without requiring changes to the scheduling logic.

C. Execution Control via Scheduler Pacing

The scheduler enforces execution order by pacing pod bindings based on their rank. After binding a pod to a node, it introduces an inherent delay, derived from the time taken to detect, score, and bind the next pod. This minimal serialization grants early-start advantages to higher-ranked jobs and reduces contention without requiring coarse-grained mechanisms. In addition, to ensure that this pacing reflects true execution progress, not just pod lifecycle states, a lightweight execution check is included: the scheduler waits for a simple signal from the currently scheduled pod once its containerized training script starts. This confirmation guards against edge cases where a pod enters the `Running` phase but is still pulling images, initializing, or allocating memory, which could otherwise allow a lower-ranked pod to jump ahead due to faster startup. The check does not impact pacing but maintains correctness under startup variability.

The scheduler also allows the pacing interval to be configured, a feature later explored in evaluation to study its impact on JCT and makespan.

IV. WORKLOAD GENERATION AND SIMULATION

A. Experimental Environment

All experiments were conducted using Minikube with the Docker driver, running a real Kubernetes cluster inside a containerized environment. The cluster was configured with 4 CPU cores and 8 GB of RAM on a single node. This setup enabled precise and repeatable control over scheduling behavior, free from multi-node variability, background workloads, or external interference.

To ensure that results reflect only the impact of scheduling logic, no resource requests or limits were specified. As

discussed earlier, static resource requests are poorly suited for ML workloads with unpredictable usage patterns, and in preliminary trials they led to pods remaining `Pending` and causing cluster instability. By avoiding them, both custom and default schedulers operated under the same assumptions, isolating the effect of execution order.

B. Workload Composition and Job Simulation

Given the defined ML features, the simulated workloads are now described based on how these features are applied, grouped into four representative categories commonly observed in real-world ML training.

- **Light jobs:** Lightweight tasks with short estimated durations, small matrices, and low gradient updates, placing minimal load on compute, memory, and communication.
- **Heavy jobs:** Compute-intensive workloads with large matrices, high gradient updates, and parallelism, resulting in high CPU and communication overhead.
- **I/O-Intensive jobs:** Jobs with frequent checkpoints and moderate size that stress the storage system through high disk I/O.
- **Slow-Converging jobs:** Jobs with low loss reduction rates and long estimated training durations, representing models that require many iterations to converge.

A Python-based workload generator programmatically creates these job definitions and embeds ML characteristics as annotations in their YAML manifests. These characteristics are passed both to the custom scheduler for ranking and to the containerized training script executed within each job, which uses them to emulate the behavior of ML model training.

The script allocates matrices of the specified size and performs repeated matrix multiplications to simulate compute-intensive operations. The total number of steps is computed proportionally to the matrix size and gradient update size, ensuring that larger jobs perform more work. Model parallelism is simulated by dividing each step into multiple partitions, with synchronization delays added to reflect inter-GPU communication overhead.

Loss reduction is modeled using an exponential decay function, and checkpoints are simulated by pausing execution at configurable intervals, with delays proportional to matrix size to reflect I/O behavior. Importantly, the simulation logic does not use the estimated training time to control when the training ends. Instead, jobs proceed through synthetic training loops until either a convergence threshold is reached (based on the loss trajectory) or a maximum step count is exceeded. This models real-world ML behavior more accurately than fixed-duration simulations, capturing effects such as early stopping, poor convergence, or prolonged training.

This design allows the simulator to mimic real ML job behavior without actual model training, making it lightweight and suitable for evaluating scheduling strategies at scale.

C. Job Submission

For comparison, two sets of identical YAML manifests were produced, one for the default scheduler and one for the custom

scheduler. These definitions are identical in configuration, ensuring fair comparison.

In both cases, all jobs were deployed simultaneously, mimicking a burst submission scenario typical in shared ML training environments. This ensured that the schedulers had to make decisions under pressure, and that any difference in performance could be attributed solely to the scheduling logic and execution order enforcement, rather than non-simultaneous job arrivals or load balancing strategies.

The default and custom scheduler evaluations were executed in separate runs on a clean cluster to avoid any cross-interference of overlapping jobs.

V. EVALUATION AND RESULTS

This section evaluates the impact of execution order and scheduling pacing on ML workload performance under different conditions. It begins by experimenting with multiple pacing configurations to understand how serialization intensity affects JCT and makespan. Based on these results, the best-performing pacing level is selected and used for all subsequent comparisons between the default Kubernetes scheduler, the ML-aware custom scheduler, and a reversed-order variant. This enables isolation and quantification of the effects of intelligent job ordering and launch timing on responsiveness, throughput, and fairness.

A. Exploring the Effect of Pacing Delay

To assess the effect of execution pacing, the 48-job half-load configuration (explained in Subsection V-B) was used and four different pacing levels were tested. The intentional delay between pod bindings in the custom scheduler was varied and three metrics were measured: the Average Inter-Launch Time (ILT), representing the time between consecutive job starts as a proxy for pacing; the Average JCT, capturing responsiveness; and the Makespan, reflecting overall throughput. The same metrics were also measured for the default Kubernetes scheduler to establish a baseline for comparison, with results summarized in Table I.

TABLE I
IMPACT OF PACING DELAY

Scheduler	Avg ILT (s)	Avg JCT (s)	Makespan (s)
Default Scheduler	0.67	230.19	443.0
Custom (delay = 0s)	5.82	165.79	410.0
Custom (delay = 1s)	6.82	180.10	427.0
Custom (delay = 2s)	7.77	196.12	442.0
Custom (delay = 5s)	10.52	254.75	528.0

These results reveal several key insights. First, the default Kubernetes scheduler exhibits minimal pacing: with an ILT of just 0.67s, it launches pods rapidly with little coordination, which leads to high contention and poor average JCT. In contrast, even without intentional delay, the scheduler introduces inherent pacing through its serialized decision loop (ILT = 5.82s), and this produces the best JCT and makespan across all variants. However, excessive pacing quickly backfires—at

a 5s delay the system becomes over-serialized, increasing idle time and pushing JCT and makespan above even the default scheduler. Makespan does not scale linearly with delay: adding 1s between 48 jobs suggests a 48s increase, but the actual impact is smaller as early completions reduce contention and free resources, compensating for the added delay. Overall, a sweet spot emerges: moderate pacing improves responsiveness and throughput, but the best results were achieved with no extra delay beyond the scheduler's inherent pacing. This confirms that controlling launch timing improves scheduling behavior, but only up to a point.

Based on these findings, the zero-delay configuration (inherent pacing only) is selected as the baseline for all subsequent evaluations, where it is compared against the default scheduler and a reversed-order variant under three job scenarios.

B. Evaluation Scenarios and Metrics

Three different job set configurations were tested to cover a range of workload intensities and system pressures:

- **12 Jobs (Normal Load):** Represents a moderately loaded system with diverse job characteristics.
- **48 Jobs (Normal Load):** Simulates a higher system load with a full batch of concurrent job submissions.
- **48 Jobs (Half Load):** Similar to the normal load scenario but with all jobs reduced in computational demand (smaller matrix sizes), testing the system under lighter resource pressure.

Each configuration was evaluated with the default Kubernetes scheduler and the custom scheduler. In addition, to isolate the effect of execution order, an ablation study was conducted using a reversed-order variant, discussed later in the results. In all cases, jobs were launched simultaneously and run to completion without CPU or memory limits.

The evaluation focuses on four key metrics: Average JCT, Makespan, Tail Latency (95th percentile JCT), and Max/Min JCT, providing a balanced view of responsiveness, throughput, and fairness. Together, these metrics provide a comprehensive view of average and worst-case performance, enabling evaluation of efficiency, fairness, and consistency across jobs.

C. Key Results and Insights

To improve statistical robustness and reduce the influence of run-to-run variation, each experiment was repeated five times under independently generated workload conditions. For each scheduler and scenario, the *mean empirical CDF (ECDF)* of JCT was then computed across runs, with an interquartile range (IQR) band to indicate variability. This smooths the distribution curves compared to single-run ECDFs, while the shaded band highlights the spread of results across trials. Summary bar charts report the mean values of key metrics (average JCT, tail latency, makespan, and JCT Min/Max) over the same five runs.

Across all three scenarios, the custom scheduler consistently outperformed the default scheduler. Average JCT was reduced by **23.8%** for 12 jobs, **22.0%** for 48 jobs (full load), and **31.6%** for 48 jobs (half load), while makespan improved by

8.6%, 3.7%, and 10.2%, respectively. These results demonstrate two important trends. First, execution-order control provides benefits regardless of workload scale: even under full 48-job pressure, the custom scheduler achieves meaningful gains over the default. Second, the improvements are magnified under lighter contention, where smarter ordering translates into proportionally larger throughput and responsiveness gains. In other words, the contribution is not only robust to scale but also especially beneficial in practical scenarios where workloads are mixed and not always cluster-saturating.

These improvements are illustrated in Figs. 2, 3, and 4, which show both the ECDFs and summary metrics for each scenario

The ablation study shows that reversing the computed order degraded JCT relative to the intended ordering by **49%** (12 jobs), **37%** (48 jobs full load), and **58%** (48 jobs half load). This demonstrates that performance depends not just on enforcing execution order, but on enforcing the *right* order, underscoring the importance of intelligent sequencing across workloads.

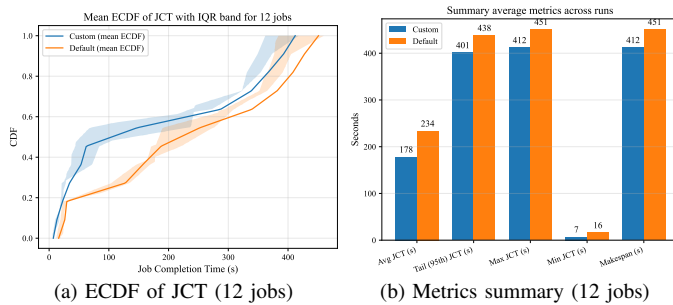


Fig. 2. 12-job scenario results.

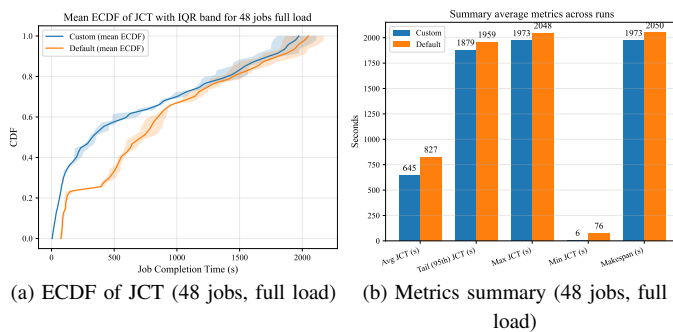


Fig. 3. 48-job full-load scenario results.

VI. CONCLUSION AND FUTURE WORK

The work in this paper introduces a custom ML-aware scheduling framework for Kubernetes, designed to address the limitations of the default scheduler in handling heterogeneous AI workloads. The system combines two key innovations: Job ranking based on ML-specific characteristics and execution pacing coupled with execution checks to control job execution order at runtime.

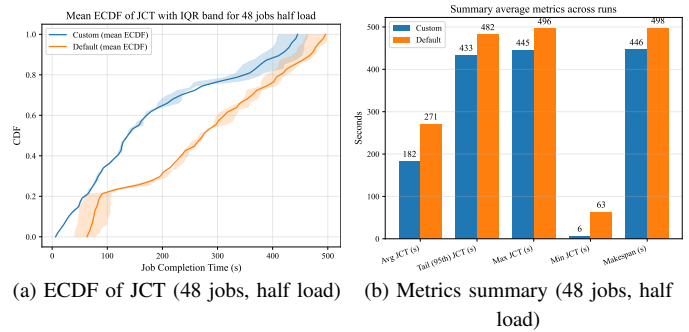


Fig. 4. 48-job half-load scenario results.

Experiments demonstrate that intelligent execution order significantly affects how workloads progress, even in unconstrained single-node settings. The pacing analysis shows that modest serialization mitigates execution-order mismatches and leverages the first-mover advantage by ensuring that higher-ranked ML jobs gain early access to compute resources. This leads to lower average JCT and smoother progression across jobs. However, excessive serialization increases makespan, confirming the need to balance controlled ordering with throughput. These findings highlight that controlling execution order, not just scheduling order, is a critical factor in efficient ML workload orchestration.

Future extensions of this work could proceed along the following directions:

- Integrating real ML training workloads (e.g., PyTorch or TensorFlow) would improve realism and enable profiling-based dynamic feature extraction for ranking.
- Combining the proposed execution-order control with existing node-selection and bin-packing strategies could enhance efficiency in multi-node heterogeneous clusters.

REFERENCES

- [1] Z. Ye, W. Gao, Q. Hu, P. Sun, X. Wang, Y. Luo, T. Zhang, and Y. Wen, "Deep learning workload scheduling in gpu datacenters: A survey," *ACM Computing Surveys*, vol. 56, no. 6, pp. 146:1–146:38, 2024.
- [2] Y. Peng, Y. Bao, Y. Chen, C. Wu, and C. Guo, "Optimus: An efficient dynamic resource scheduler for deep learning clusters," in *Proceedings of the Thirteenth EuroSys Conference*, 2018, pp. 1–14.
- [3] P. Naayini, *Building AI-Driven Cloud-Native Applications with Kubernetes and Containerization*, 2024.
- [4] J. Ray, "Taming the memory beast: Strategies for reliable ml training on kubernetes," *arXiv preprint*, vol. arXiv:2412.14701, 2024.
- [5] Kubernetes, "Kubernetes scheduler documentation," 2024, <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>.
- [6] J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, and C. Guo, "Tiresias: A gpu cluster manager for distributed deep learning," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2019, pp. 485–500.
- [7] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, and L. Zhou, "Gandiva: Introspective cluster scheduling for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 595–610.
- [8] C. Carrión, "Kubernetes scheduling: Taxonomy, ongoing issues and challenges," *ACM Computing Surveys*, vol. 55, no. 7, pp. 138:1–138:37, 2023.
- [9] Z. Rejiba and J. Chamanara, "Custom scheduling in kubernetes: A survey on common problems and solution approaches," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–37, 2022.

- [10] Kubernetes, “Pod lifecycle documentation,” 2024, <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>.
- [11] J. P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, “The linux scheduler: A decade of wasted cores,” in *Proceedings of the Eleventh European Conference on Computer Systems*, 2016, pp. 1–16.
- [12] The Linux Kernel Developers, “Cfs scheduler — the linux kernel documentation,” 2025, <https://docs.kernel.org/scheduler/sched-design-CFS.html>.
- [13] Linux Documentation Project, “Processes,” 2024, <https://tldp.org/LDP/tlk/kernel/processes.html>.
- [14] SUSE LLC, “15 tuning the task scheduler,” 2025, <https://documentation.suse.com/es-es/sles/15-SP6/html/SLES-all/cha-tuning-taskscheduler.html>.
- [15] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, “Dominant resource fairness: Fair allocation of multiple resource types,” in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2011.
- [16] H. Zhang, M. Chowdhury, J. M. Hellerstein, J. E. Gonzalez, and I. Stoica, “Slaq: Quality-driven scheduling for distributed machine learning,” in *ACM Symposium on Operating Systems Principles (SOSP)*, 2017.
- [17] X. Qiu, D. Narayanan, A. Phanishayee, S. Agarwal, and P. Ranganathan, “Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning,” in *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021.
- [18] Volcano Community, “Volcano: A Batch System on Kubernetes,” 2024, <https://volcano.sh>.
- [19] Kueue Maintainers, “Kueue: Kubernetes-native Job Queueing,” 2024, <https://kueue.sigs.k8s.io>.
- [20] Argo Project, “Argo Workflows Documentation,” 2025, <https://argo-workflows.readthedocs.io/en/latest/>.
- [21] Kubeflow Community, “Kubeflow Pipelines Overview,” 2025, <https://www.kubeflow.org/docs/components/pipelines/overview/>.